test_first/04/app/views/projects/index.html.erb

```
<h1>All Projects</h1>
<table>
  <thead>
    <tr>
      <td>Project Name</td>
      <td>Total Project Size</td>
    </tr>
  </thead>
  <tbody>
    <% @projects.each do |project| %>
      <tr>
        <td><%= project.name %></td>
        <td><%= project.total_size %></td>
      </tr>
    <% end %>
  </tbody>
</table>
```

Don't be concerned with making this pretty; in a production application, presumably the markup would be more complex.

When this executes, your newly entered project gets its own table row—complete with its name and size.

And your end-to-end test finally passes.

That puts you in a refactoring phase. You didn't write much code in this step, but I'd like to take the opportunity to refactor the last couple of lines of the end-to-end test using the Capybara-Rails has_selector matcher. This is a really common work pattern for me. Sometimes I have trouble seeing the shape of a view before I write it, so I write a very loose test and then tighten the test once I see what pieces of the view will exist for me to hook onto.

The has_selector method takes as its argument a jQuery-style selector, with the usual # representing a DOM ID, and a dot (.) representing a DOM class. The assertion passes if the page contains a DOM element that matches the selector. You can also specify a text: option that means the matching DOM element must also contain particular text (or match a particular regular expression).

With has_selector, you can rewrite the test as follows: